📘 Theoretical Definition

> A **context manager** in Python is a construct that allows you to **manage resources safely and efficiently** using the `with` statement. It works by defining the special methods `__enter__` and `__exit__`.

✅ Benefits

- Automatically handles **resource cleanup** (e.g., closing files or database connections).
- Produces **cleaner, more readable code**.
- Provides **robust error handling**.

🧪 Classic Example with Files:

python Copiar Editar

```python
with open("example.txt", "w") as f:
    f.write("Hello, world!")

# The file is automatically closed when exiting the block
```

Internally, this is equivalent to:

python Copiar Editar

```python
f = open("example.txt", "w")
try:
    f.write("Hello, world!")
finally:
    f.close()
```

Creating a Custom Context Manager

You can create your own context manager by defining `__enter__` and `__exit__` in a class:

python Copiar Editar

```python
class MyContextManager:
    def __enter__(self):
        print("Entering context...")
        return self

    def __exit__(self, exc_type, exc_value, traceback):
        print("Exiting context...")
        if exc_type:
            print(f"An error occurred: {exc_value}")
        return False  # Return True to suppress the exception

# Using the context manager
with MyContextManager() as manager:
    print("Inside the context")
    # raise ValueError("Oops!")  # Uncomment to test error handling
```

🔍 What Do `__enter__` and `__exit__` Do?

- `__enter__`: Called at the **start** of the `with` block. The return value is assigned to the variable after `as`.
- `__exit__`: Called when exiting the block, either **normally or due to an exception**. If it returns `True`, it suppresses the exception.

🧰 Alternative: Using `contextlib`

Python also allows simpler context managers using a decorator:

python Copiar Editar

```python
from contextlib import contextmanager

@contextmanager
def my_context():
    print("Entering")
    yield
    print("Exiting")

with my_context():
    print("Doing stuff...")
```